



Department of Computer Science and Engineering

Academic year 2024-2025 (Odd Semester)

CIE III: Question Paper

Course	Operating Systems		
Date	JAN 2026	Maximum Marks	60
Course Code	CS235AI	Duration	120 Min
Sem	III	CIE – III	

SL No.	Part A	M	BT	CO
1.1	Formulate how long a paged memory reference takes if memory reference takes 200 nanoseconds. Assume a paging system with page table stored in memory. <u>Ans:</u> 400 nanoseconds. 200 ns to access the page table plus 200 ns to access the word in memory.	01	2	2
1.2	Consider a logical address space of 64 pages of 1024 words each, mapped onto a physical memory of 32 frames. a. How many bits are there in the logical address? b. How many bits are there in the physical address? a. Logical address: 16 bits Physical address: 15bits	02	1	2
1.3	What is compaction? Example Compaction is the process of rearranging memory contents to place all free memory together, reducing external fragmentation. Example: If memory has allocated blocks A, free space B, free space, compaction moves B next to A to create a larger free block.	02	1	1
1.4	In the working set model, for: 2 6 1 5 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3, if Delta =10, then the working set at time t1(...751) is what? <u>Identify the last Δ=10 Delta</u> The last 10 page references up to and including time t1t_1t1 are: Pages referenced in 14–23: 4, 1, 2, 3, 4, 4, 4, 3, 4, 3 Unique pages: 1, 2, 3, 4	02	3	1
1.5	A file of size 80 KB is stored on a disk with block size 1 KB. How many blocks are required? An index block can store 256 pointers. Block size is 4 KB. What is the maximum file size? Number of blocks = 80 KB / 1 KB = 80 blocks Maximum file size = 256 × 4 KB = 1024 KB (1 MB)	02	3	2
1.6	How OS utilizes swapping technique Swapping moves entire processes between main memory and disk to free memory or allow multitasking. It involves saving process state, moving it to	01	1	2

	disk, and restoring it later.			
	Part B			
1	Consider the following page reference string with 3 page frames: 1, 2, 3, 4, 2, 1, 5, 6, 1, 2, 3, 7, 6, 3, 2, 1, 3, 6 a) Determine the number of page faults using the FIFO page replacement algorithm. b) Determine the number of page faults using the LRU page replacement algorithm. Assume that the frames are initially empty. Show all steps clearly. c) Which algorithm performs better for this reference string? Justify your answer. a) FIFO Counting page faults: 1 (fault), 2 (fault), 3 (fault), 4 (fault, replace 1), 2 (hit), 1 (fault, replace 2), 5 (fault, replace 3), 6 (fault, replace 4), 1 (hit), 2 (fault, replace 5), 3 (fault, replace 6), 7 (fault, replace 1), 6 (fault, replace 2), 3 (hit), 2 (fault, replace 7), 1 (fault, replace 3), 3 (fault, replace 6), 6 (fault, replace 2) - Total page faults = 15 b) LRU: - Track least recently used page. - Counting page faults: 1 (fault), 2 (fault), 3 (fault), 4 (fault), 2 (hit), 1 (hit), 5 (fault), 6 (fault), 1 (hit), 2 (hit), 3 (fault), 7 (fault), 6 (hit), 3 (hit), 2 (hit), 1 (hit), 3 (hit), 6 (hit) - Total page faults = 12	4+4+2	3	3
2.a	Given a system that runs large database applications and modular software systems, analyze whether paging or segmentation is more suitable. Justify your choice by discussing fragmentation, protection mechanisms, and logical program structure. For a system running large database applications and modular software systems, paging is generally more suitable for database workloads. Paging divides memory into fixed-size pages, which eliminates external fragmentation, making it easier to allocate memory efficiently for large datasets. Although paging can cause internal fragmentation, the impact is minimal with appropriately chosen page sizes. Additionally, paging provides consistent and predictable memory management, which is crucial for high-performance database operations that require frequent memory access. Segmentation is more aligned with modular software systems, as it supports logical program divisions such as code, data, and stack segments. Segmentation provides per-segment protection and sharing, which is useful for software modules that require isolation and controlled access. However, it suffers from external fragmentation and more complex memory allocation. Therefore, for a system combining both types of workloads, a hybrid	03+3	2	2

	approach—paging within segments—can be ideal, offering the memory efficiency of paging and the logical protection of segmentation.			
2.b	A company is designing a memory management system for its new operating system. The system will run: 1. A database server that frequently stores large, fixed-size records. 2. A modular software development environment with multiple independent modules loaded and unloaded dynamically. The system designer is debating between paging and segmentation. a) Explain which type of fragmentation (internal or external) is more likely to occur in each case and why. b) If the database grows continuously and modules are loaded/unloaded frequently, analyse how each fragmentation type would affect memory utilization and system performance. For the database server with large, fixed-size records, paging is suitable, and internal fragmentation is likely because the last page of each record may not be fully utilized. For the modular software environment, segmentation fits better, and external fragmentation occurs as variable-sized modules are loaded and unloaded, leaving scattered free memory blocks. Internal fragmentation slightly reduces memory utilization but keeps allocation simple and predictable. External fragmentation can significantly reduce usable memory and may require compaction or complex memory management. To mitigate these issues, choose an appropriate page size for the database and consider a paged segmentation (hybrid) approach for modular programs to combine efficient memory use with logical program protection.	02+2	2	2

3.a	A system has memory partitions of sizes: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB. Four processes with sizes 212 KB, 417 KB, 112 KB, 426 KB need to be allocated memory. Tasks: a) Allocate the processes to memory partitions using the following allocation strategies: • First-Fit • Best-Fit • Worst-Fit b) Show the allocation step-by-step and indicate any processes that cannot be allocated. c) Analyze which allocation strategy is most efficient in terms of memory utilization and briefly justify your answer. First-Fit Allocation P4 cannot be allocated as no remaining partition is large enough. <table><thead><tr><th>Process</th><th>Size</th><th>Allocated Partition</th><th>Remaining Partition Size</th></tr></thead><tbody><tr><td>P1</td><td>212</td><td>500 KB</td><td>288 KB</td></tr><tr><td>P2</td><td>417</td><td>600 KB</td><td>183 KB</td></tr><tr><td>P3</td><td>112</td><td>200 KB</td><td>88 KB</td></tr><tr><td>P4</td><td>426</td><td>None</td><td>—</td></tr></tbody></table> <table><thead><tr><th>Process</th><th>Size</th><th>Allocated Partition</th><th>Remaining Partition Size</th></tr></thead><tbody><tr><td>P1</td><td>212</td><td>300 KB</td><td>88 KB</td></tr><tr><td>P2</td><td>417</td><td>500 KB</td><td>83 KB</td></tr><tr><td>P3</td><td>112</td><td>200 KB</td><td>88 KB</td></tr><tr><td>P4</td><td>426</td><td>600 KB</td><td>174 KB</td></tr></tbody></table> Best fit: All processes are allocated. Minimizes leftover space → most efficient allocation. Worst Fit P4 cannot be allocated. Leaves large partitions unused → less memory-efficient than Best-Fit. <table><thead><tr><th>Process</th><th>Size</th><th>Allocated Partition</th><th>Remaining Partition Size</th></tr></thead><tbody><tr><td>P1</td><td>212</td><td>600 KB</td><td>388 KB</td></tr><tr><td>P2</td><td>417</td><td>500 KB</td><td>83 KB</td></tr><tr><td>P3</td><td>112</td><td>300 KB</td><td>188 KB</td></tr><tr><td>P4</td><td>426</td><td>None</td><td>—</td></tr></tbody></table>	Process	Size	Allocated Partition	Remaining Partition Size	P1	212	500 KB	288 KB	P2	417	600 KB	183 KB	P3	112	200 KB	88 KB	P4	426	None	—	Process	Size	Allocated Partition	Remaining Partition Size	P1	212	300 KB	88 KB	P2	417	500 KB	83 KB	P3	112	200 KB	88 KB	P4	426	600 KB	174 KB	Process	Size	Allocated Partition	Remaining Partition Size	P1	212	600 KB	388 KB	P2	417	500 KB	83 KB	P3	112	300 KB	188 KB	P4	426	None	—	03+3+4	2	2
Process	Size	Allocated Partition	Remaining Partition Size																																																													
P1	212	500 KB	288 KB																																																													
P2	417	600 KB	183 KB																																																													
P3	112	200 KB	88 KB																																																													
P4	426	None	—																																																													
Process	Size	Allocated Partition	Remaining Partition Size																																																													
P1	212	300 KB	88 KB																																																													
P2	417	500 KB	83 KB																																																													
P3	112	200 KB	88 KB																																																													
P4	426	600 KB	174 KB																																																													
Process	Size	Allocated Partition	Remaining Partition Size																																																													
P1	212	600 KB	388 KB																																																													
P2	417	500 KB	83 KB																																																													
P3	112	300 KB	188 KB																																																													
P4	426	None	—																																																													
4.a	Discuss situation under which the most frequently used page replacement algorithm generates fewer page faults than the least frequently used page replacement algorithm. Also discuss under which circumstances the opposite holds. The Most Frequently Used (MFU) page replacement algorithm generates	06	2	2																																																												

	fewer page faults than Least Frequently Used (LFU) when a program exhibits phase changes or when pages that were heavily accessed in the past are no longer needed in the future. In such situations, LFU continues to retain pages with high access counts, assuming they are still important, even though their usefulness has ended. MFU, on the other hand, replaces these high-frequency but obsolete pages, making room for newly required pages and thereby reducing page faults. This behavior is common in programs with initialization phases or changing locality of reference. The opposite occurs when access patterns are stable, and past frequency is a good indicator of future use. In workloads such as loops or database applications, pages that are frequently accessed tend to remain important throughout execution. In these cases, LFU performs better by keeping high-frequency pages in memory, resulting in fewer page faults. MFU performs poorly here because it may evict frequently used pages that are still needed, increasing the number of page faults.			
4.b	A system uses copy-on-write for virtual memory. Analyze a scenario where multiple processes share a large read-only dataset. How does COW improve memory utilization in this case? When multiple processes share a large read-only dataset, copy-on-write (COW) allows them to use the same physical pages without duplication, marking them as read-only. A private copy is created only if a process modifies a page, minimizing memory usage while maintaining process isolation.	04	2	1
5.a	Discuss the role of various file attributes in operating systems, explaining how they contribute to security, system auditing, recovery, and efficient file operations. In your answer, include ownership, permissions, timestamps, file size, location, and extended attributes. File attributes are metadata associated with a file that the operating system maintains to manage and protect data efficiently. - Security Enforcement: - Ownership and permission attributes define who can read, write, or execute a file. - These attributes prevent unauthorized access and ensure isolation in multi-user systems. - System Auditing and Recovery: - Timestamps (creation, last modification, last access) track file usage patterns. - They help detect intrusions, monitor activity, and assist in backup and recovery operations. - Efficient File Operations: - File size and location attributes allow the OS to calculate block addresses, allocate memory buffers, and enforce read/write boundaries. - These attributes are essential for sequential and random file access, ensuring correct data retrieval and storage. - Extended Attributes: - Store additional metadata such as Access Control Lists (ACLs), security labels, or application-specific information. - They support advanced security mechanisms, policy enforcement, and enhanced file management.	06	3	2

5.b	Compare sequential, direct, and indexed sequential file access methods in terms of efficiency and use cases. Clarify why sequential access remains widely used, why indexed sequential is preferred in databases, and the efficiency differences between linked and indexed allocation for random access. Sequential access remains widely used for large-scale data processing due to simplicity and minimal seek overhead. Indexed sequential access is optimal for databases, balancing fast lookups and sequential traversal. Allocation methods significantly impact the efficiency of random access, with indexed allocation outperforming linked allocation for direct retrieval tasks.	04	2	3
6	How does demand paging enable efficient memory usage in real-world systems such as multitasking operating systems, and what hardware support is required to make demand paging possible? A program has 10 pages but only 4 page frames are available. Explain how demand paging decides which pages to load into memory when a page fault occurs. Demand paging: Pages are loaded into memory only when needed (on page fault), reducing memory usage. Hardware support: • Page table with valid/invalid bits. • MMU for address translation. • Page fault interrupt. -Page Fault Triggered: The CPU tries to access a page (from the 10 total) not currently in any of the 4 physical memory frames, causing a page fault. OS Intervention: Control transfers to the Operating System (OS). Locate Page: The OS finds the required page in secondary storage (disk). Check for Free Frame: If Free Frame Exists: The OS loads the requested page into the free frame, updates the page table (marking the page as valid), and restarts the instruction. If No Free Frame (Memory Full): A page replacement algorithm is invoked. Page Replacement (If Needed): Algorithm Selection: The OS uses a strategy (e.g., Least Recently Used (LRU), First-In, First-Out (FIFO)) to pick a page to evict, called the "victim". Eviction: If the victim page is "dirty" (modified), it's written back to disk; otherwise, it's just discarded. Load New Page: The newly demanded page is loaded into the now-free frame. Update Tables: The page tables are updated for both the evicted and newly loaded pages. Resume Execution: The process continues from where it left off.	6+4	4	4

Course Outcomes: After completing the course, the students will be able to:-	
CO 1	Demonstrate the fundamental concepts of operating system like process management, file management, memory management and issues of synchronization.
CO 2	Analyze and interpret operating system concepts to acquire a detailed understanding of the course.

CO 3	Apply the operating systems concepts to address related new problems in computer science Domain.
CO 4	Design or develop solutions using modern tools to solve applicable problems in operating systems domain.
CO5	Extend the theoretical knowledge acquired through the course to demonstrate skills like investigation, effective communication, working in team/Individual, following ethical practices by implementing operating system concepts/ applications and engage in life long learning.

BT-Blooms Taxonomy, CO-Course Outcomes, M-Marks

Marks Distribution	Particulars	CO1	CO2	CO3	CO4	CO5	L1	L2	L3	L4	L5	L6
	Test	3	17	26	14	-	3	27	20	10	-	-